

第 0 卷 作战流程与题型路由

考试速查：先扫三题、判断复杂度、选择题型路由、决定先交暴力还是直接套模板；这里是全套资料的总导航。

Contents

第 0 卷 作战流程与题型路由	2
考试速查定位	2
第 0 卷组速查	2
常见路由名到模块编号	2
OPS-00 第 -1 卷：统一接口与标准容器	3
0. 考场使用顺序	3
1. 全局 C++17 骨架	3
2. 函数命名规范	4
3. Graph 协议	5
4. Array 协议	6
5. Query 协议	7
6. Compressor 协议	8
7. State 协议	8
8. 模块契约卡	9
9. 最小自检	10
OPS-01 第 0 卷：考场作战流程	10
1. 考场总原则	10
2. 3 小时时间策略	11
3. 32 次提交策略	11
4. 先交部分分再升级	11
5. 提交前检查清单	12
6. 完全不会时兜底策略	12
7. 卡住时的 5 分钟复位	13
ROUTE-00 第 0A 卷：题面路由表	13
1. 3 分钟路由动作	14
2. 题面信号路由表	14
3. 数据范围预算表	15
4. 操作类型路由表	16
5. 路由冲突时怎么判	16
ROUTE-01 第 0A 卷：拼接食谱	16
食谱格式	17
R01 静态区间和	17
R02 单点修改 + 区间和	18
R03 区间加 + 最后输出	18
R04 区间加 + 单点查	19
R05 区间加 + 区间和	19
R06 静态区间最值	19
R07 大坐标动态统计	20
R08 逆序对	20
R09 滑动窗口最值	21
R10 无权最短路	21
R11 非负权最短路	22
R12 小图全源最短路	22
R13 最小生成树	23
R14 DAG 依赖顺序 + DP	23
R15 树上路径查询	23

R16 0/1 背包	24
R17 区间 DP	24
R18 访问所有点状压	25
R19 复杂状态 DFS + memo	25
R20 二分答案 + 检查函数	26
v0.2 本卷例题训练区	26
V00-EX01 静态区间和路由	26
V00-EX02 离线区间加最终数组	27
V00-EX03 单点赋值与区间和	28
V00-EX04 大坐标频次统计	30
V00-EX05 网格最少步数	32
V00-EX06 非负权道路最短路	33
V00-EX07 依赖任务排序	34
V00-EX08 容量选择路线	36
V00-EX09 模式串出现次数	37
V00-EX10 提交版本选择器	38
第 7 卷: 调试、反例与对拍训练	39
V00-CEX01 数据范围路由卡	39
V00-CEX02 先交差分部分分	40
V00-CEX03 BFS 与 Dijkstra 路由合并	41
V00-CEX04 小数据暴力与大数据特判	42
V00-CEX05 最高分提交统计	43

第 0 卷 作战流程与题型路由

考试速查定位

第 0 卷 作战流程与题型路由

- **这是第几卷：** 00。
- **主要查什么：** 先扫三题、判断复杂度、选择题型路由、决定先交暴力还是直接套模板；这里是全套资料的总导航。
- **什么时候翻：** 考试开始、读题后不知道翻哪里、需要复杂度/提交策略/统一协议时先翻。
- **题面关键词：** 时间分配；复杂度表；题型路由；模块拼接；1-index；提交检查。

自动由 OPS/ROUTE 模块重建。定位是统一协议、题型路由、考场时间分配和提交检查。

第 0 卷组速查

分区	用途
OPS-*	考场流程、提交策略、检查清单
ROUTE-*	题型信号到模块组合的总路由

常见路由名到模块编号

路由名	模块入口
PrefixSum / Difference	DS-01
树状数组 / 双树状数组	DS-02
SegmentTree / SparseTable	DS-03
TwoPointers / SlidingWindow	DS-06
DSU / Kruskal	DS-04 或 GRAPH-06
Graph / BFS / Dijkstra	GRAPH-00/02/03
Topo / DAG DP	GRAPH-05
LCA / Tree	GRAPH-09 / TREE-*
Knapsack / LIS / LCS	DP-06/07/08/11/23/24
String matching	STR-02 / CPP-011

OPS-00 第 -1 卷：统一接口与标准容器

模块编号：OPS-00

模块名称：第 -1 卷 统一接口与标准容器

标签：前置卷、接口协议、标准容器、C++17、考场装配

一句话用途：把题面输入整理成统一形状，让后面的图论、数据结构、DP、暴力模块可以直接拼上去。

题面触发词：任意题目开写前都先看本卷。

什么时候用：每道题开始前，用本卷确定索引、类型、容器、函数名和模块调用外壳。

不要什么时候用：不要把本卷当算法教材；它只规定“零件接口”，不解释算法原理。

复杂度：本卷本身没有算法复杂度；复杂度由下游模块决定。

数据范围参考：所有规模都适用，尤其适合需要快速拼接多个模块的题。

依赖的标准容器：无。

输入如何整理：先读题面原始输入，再转为 1-index 全局数组、Graph、Query、State、Compressor。

接口：init/build/add/add_edge/setv/range_add/query/prefix/at/solve_xxx。

输出能力：提供统一外壳和下游模块调用约定。

下游可接：PrefixSum、树状数组、SegmentTree、DSU、BFS、Dijkstra、Topo、DP、DFS memo。

可拼接模块：所有模块。

模板代码：见本文件各协议代码块。

调用示例：见各协议的“考场抄法”。

常见坑：混用 0-index/1-index、区间半开半闭、图边重复、把 int 当距离、函数直接输出导致无法升级。

暴力/部分替代：统一保留 solve_bruteforce()，先拿小数据分，再替换 solve_optimized()。

升级方向：把暴力外壳升级为 memo、DP、图论或数据结构正解，但输入整理层不改。

最小测试样例：每个模块接入后都测单点、边界、空结果、重复元素、极值。

0. 考场使用顺序

1. 读题，圈出 n/m/q、是否有修改、是否是图/树/区间/状态。
2. 按本卷确定标准容器：Graph / Array / Query / State。
3. 把输入只整理一次，不在算法里边读边算。
4. 先写 solve_bruteforce() 或最简单合法版本。
5. 再把核心函数替换成正解模块。
6. 提交前只检查接口契约：索引、区间、类型、方向、初始化。

1. 全局 C++17 骨架

每份代码开头统一抄这一段。不要加考试环境不一定支持的优化开关。

如果现场 GNU g++ 支持 bits/stdc++.h，直接用短版。若不支持，把第一行替换成 CPP-001 的“标准头文件安全包”，不要现场逐个猜缺哪个头。

```
#include <bits/stdc++.h>
using namespace std;

using ll = long long;
using pii = pair<int, int>;
using pll = pair<ll, ll>;

const int INF = 1000000000;
const ll LINF = 4'000'000'000'000'000'000LL;
const ll MOD = 1000000007LL; // 题目给别的模数就改这里

void solve() {
    // write here
}
```

```
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    solve();
    return 0;
}
```

硬规则:

项目	统一约定
C++ 标准	C++17
禁止	#pragma GCC optimize、freopen、文件 IO、#define int long long
点编号	默认 1..n
数组/DP 表	默认 1-index; 上限明确时优先全局静态数组 a[MAXN] / dp[MAXN][MAXM]
区间	默认闭区间 [l, r]
字符串	默认 C++ 自然 0-index, 进入字符串模块时单独提醒
网格	默认 1..n, 1..m, 转图时用 id(i, j)
权值/距离/答案	优先 long long
数量/下标	一般 int

数组选择口令:

题目给明确上限: 优先全局静态数组, 编号 1..n, 防御性多开 5~10。

输入规模不固定或需要可变长度: 再用 vector, 并主动开 n+1 保持 1-index。

字符串、queue、deque、priority_queue、set/map、unordered_map 这类容器行为: 直接用 STL。

集合状压的 mask 数值必须从 0 枚举, 但业务对象编号仍写 1..k, 取位统一写 i-1。

2. 函数命名规范

统一动词让模块能替换。

函数名	用途	例子
init(...)	清空并重新初始化	fw.init(n)、G.init(n)
build(...)	从已有数组/图建立结构	ps.build(a)、st.build(a)
add(pos, val)	单点增加	树状数组单点加
add_undirected/add_directed	图加边	G.add_undirected(u, v, w)
setv(pos, value)	单点赋值	线段树点赋值, 避免撞 std::set
range_add(l, r, val)	闭区间加	Lazy Segment Tree
query(l, r)	闭区间查询	和/最值/gcd
prefix(pos)	查询 [1, pos]	前缀和、树状数组
at(pos)	单点查询	差分树状数组
solve_bruteforce()	小数据精确版	部分分
solve_memo()	记忆化版	从 DFS 升级
solve_optimized()	正解版	DP/图论/数据结构

接口原则:

算法函数尽量返回结果, 不直接 cout。

solve() 负责读入、选择版本、输出。

标准外壳:

```
void read_input() {
    // 只读入并整理成标准容器
}
```

```
ll solve_bruteforce() {
    // 小数据精确版
    return 0;
}
```

```

}

ll solve_optimized() {
    // 正解版
    return 0;
}

void solve() {
    read_input();
    cout << solve_optimized() << "\n";
}

```

3. Graph 协议

目标：一份建图同时服务 BFS、DFS、Dijkstra、Topo、Kruskal、Floyd、Bellman-Ford、LCA。

```

struct AdjEdge {
    int to;
    ll w;
    int edge_index; // 内部边下标, 只给模板内部用
    bool directed;
    int input_id;    // 对外边号, 1-index
};

struct FullEdge {
    int from, to;
    ll w;
    bool directed;
    int input_id;    // 对外边号, 1-index
};

struct Graph {
    int n;
    vector<vector<AdjEdge>> g;
    vector<FullEdge> edges;

    Graph(int n = 0) { init(n); }

    void init(int n_) {
        n = n_;
        g.assign(n + 1, {});
        edges.clear();
    }

    void add_edge_raw(int u, int v, ll w, bool directed) {
        int edge_index = (int)edges.size(); // 内部 0-based, 不输出
        int input_id = edge_index + 1;      // 题面边号 1-based
        edges.push_back({u, v, w, directed, input_id});
        g[u].push_back({v, w, edge_index, directed, input_id});
        if (!directed) g[v].push_back({u, w, edge_index, directed, input_id});
    }

    void add_undirected(int u, int v, ll w = 1) {
        add_edge_raw(u, v, w, false);
    }

    void add_directed(int u, int v, ll w = 1) {
        add_edge_raw(u, v, w, true);
    }
};

```

考场抄法:

```

int n, m;
cin >> n >> m;

```

```

Graph G(n);
for (int i = 1; i <= m; i++) {
    int u, v;
    ll w;
    cin >> u >> v >> w;
    G.add_undirected(u, v, w); // 无向图
}

```

无权图:

```

int u, v;
cin >> u >> v;
G.add_undirected(u, v); // 无向无权, 推荐写法
G.add_directed(u, v);   // 有向无权, 推荐写法

```

Graph 的双视图:

视图	用途	遍历方式
G.g[u]	BFS、DFS、Dijkstra、Topo、SCC、LCA	for (auto e : G.g[u])
G.edges	Kruskal、Floyd 初始化、Bellman-Ford、全边排序	for (auto e : G.edges)

Graph 接口坑位:

1. Kruskal 只遍历 G.edges, 不遍历 G.g, 否则无向边会重复。
2. Bellman-Ford 遇到无向边要松弛两个方向。
3. Topo 只适用于 `G.add\_directed(u, v)` 建出的有向边。
4. LCA/树 DP 要用无向边, 并从根 DFS。
5. 最大流不要用 Graph, 另用 FlowGraph。
6. 考场只写 add_undirected/add_directed, 不直接调用 add_edge_raw。
7. 点编号默认 1-index; 对外边号用 input_id, 也是 1-index。edge_index 是内部跳父边用的下标, 不输出。

4. Array 协议

标准读入:

```

int n;
cin >> n;
vector<ll> a(n + 1);
for (int i = 1; i <= n; i++) cin >> a[i];

```

区间约定:

所有区间都是闭区间 $[1, r]$ 。

空区间 $l > r$ 时, 查询类函数返回 0 或对应单位元。

静态前缀和协议:

```

struct PrefixSum {
    int n;
    vector<ll> pre;

    void build(const vector<ll> &a) {
        n = (int)a.size() - 1;
        pre.assign(n + 1, 0);
        for (int i = 1; i <= n; i++) pre[i] = pre[i - 1] + a[i];
    }

    ll prefix(int pos) const {
        if (pos <= 0) return 0;
        if (pos > n) pos = n;
        return pre[pos];
    }

    ll query(int l, int r) const {

```

```

        if (l > r) return 0;
        return prefix(r) - prefix(l - 1);
    }
};

```

动态区间和协议:

```

struct BIT {
    int n;
    vector<ll> bit;

    void init(int n_) {
        n = n_;
        bit.assign(n + 1, 0);
    }

    void add(int pos, ll val) {
        if (pos <= 0 || pos > n) return;
        for (; pos <= n; pos += pos & -pos) bit[pos] += val;
    }

    void build(const vector<ll> &a) {
        init((int)a.size() - 1);
        for (int i = 1; i <= n; i++) add(i, a[i]);
    }

    ll prefix(int pos) const {
        pos = min(pos, n);
        ll res = 0;
        for (; pos > 0; pos -= pos & -pos) res += bit[pos];
        return res;
    }

    ll query(int l, int r) const {
        if (l > r) return 0;
        return prefix(r) - prefix(l - 1);
    }

    ll at(int pos) const {
        return query(pos, pos);
    }
};

```

线段树统一接口先记名字, 代码放数据结构卷:

```

build(a);
add(pos, delta);
setv(pos, value);
range_add(l, r, delta);
query(l, r);

```

5. Query 协议

多次询问统一先存成 Query, 不要把各种题写成各自的临时变量。

```

struct Query {
    int l = 0, r = 0;
    int id = 0;
    int op = 0;    // 题目有操作类型时使用
    int pos = 0;   // 单点位置
    ll x = 0;      // 修改值、阈值、权值
};

```

常见读法:

```

int q;
cin >> q;

```

```
vector<Query> qs(q + 1);
for (int i = 1; i <= q; i++) {
    cin >> qs[i].op >> qs[i].l >> qs[i].r;
    qs[i].id = i;
}
```

离线查询约定:

id 保留原顺序。

排序只排序 Query 数组, 不改答案数组。

答案统一放 ans[id]。

6. Compressor 协议

值域很大但实际出现的数不多时, 先压缩再接 树状数组/Segment Tree。

```
struct Compressor {
    vector<ll> xs;

    void build(vector<ll> v) {
        xs = v;
        sort(xs.begin(), xs.end());
        xs.erase(unique(xs.begin(), xs.end()), xs.end());
    }

    int id(ll x) const {
        return lower_bound(xs.begin(), xs.end(), x) - xs.begin() + 1;
    }

    int lower_id(ll x) const {
        return lower_bound(xs.begin(), xs.end(), x) - xs.begin() + 1;
    }

    int upper_id(ll x) const {
        return upper_bound(xs.begin(), xs.end(), x) - xs.begin();
    }

    ll val(int pos) const {
        return xs[pos - 1];
    }

    int size() const {
        return (int)xs.size();
    }
};
```

接法:

```
vector<ll> all;
// 把所有 a[i]、查询里的 x、区间端点等需要比较的值放进 all
Compressor cp;
cp.build(all);

BIT fw;
fw.init(cp.size());
fw.add(cp.id(x), 1);
```

压缩坑位:

1. 压缩后编号仍然从 1 开始。
2. x 一定出现过, 用 id(x)。
3. 查询原坐标范围 [L, R], 用 lower_id(L), upper_id(R)。
4. lower_id(L) > upper_id(R) 表示范围内没有点。

7. State 协议

状态统一拆成三类:

State = {进度, 资源, 记忆}

类别	常用变量名	例子
进度	i, pos, u, step	处理到第几个物品、当前点
资源	cap, k, rem, cost	剩余容量、次数、预算
记忆	mask, last, color, used	已访问集合、上一个点、颜色

纸质默认稳妥写法:

```
map<tuple<int, int, int>, ll> memo;
```

```
ll dfs(int i, int cap, int mask) {
    if (i == n + 1) return 0; // base case 按题意替换
    auto key = make_tuple(i, cap, mask);
    if (memo.count(key)) return memo[key];

    ll ans = -LINF;
    // 枚举选择
    return memo[key] = ans;
}
```

边界清晰时换成数组:

```
const int MAXN = 200000 + 5;
const int MAXW = 5000 + 5;
static ll dp[MAXN][MAXW];

for (int i = 0; i <= n; i++) {
    for (int j = 0; j <= W; j++) dp[i][j] = -LINF;
}
```

状态存储选择:

情况	容器	考场优先级
范围小且明确	vector / 多维 vector	最高
有集合访问	vector + mask	高
维度复杂但状态少	map<tuple<...>, ll>	最稳
需要速度且能编码	unordered_map<ll, ll>	升级

8. 模块契约卡

每个算法模块接入前, 先在纸上补全这张卡。

模块名:

输入依赖:

输出能力:

下游可接:

不能处理:

初始化顺序:

查询/更新接口:

复杂度:

最小测试:

例: Dijkstra

输入依赖: Graph, 非负边权, 起点 s

输出能力: dist[i]

下游可接: 路径恢复、最短路 DAG、计数 DP

不能处理: 负权边

初始化顺序: 建图 -> dist=LINF -> pq

查询/更新接口: vector<ll> dijkstra(const Graph& G, int s)

复杂度: $O((n+m)\log n)$

最小测试: 3 点链、不可达点、重边、起点到自己

9. 最小自检

每次拼完模块，提交前用 60 秒过这张表。

检查项	问自己
索引	输入是 0-index 还是 1-index? 我有没有统一?
区间	所有 [1, r] 都是闭区间吗?
类型	距离、答案、乘法有没有用 ll?
初始化	多组数据时数组、图、memo 有没有清空?
图方向	directed 写对了吗?
不可达	LINF 会不会被拿去加法溢出?
排序	离线排序后答案有没有按 id 放回?
输出	每个查询都有输出吗? 换行够吗?

OPS-01 第 0 卷：考场作战流程

模块编号：OPS-01

模块名称：第 0 卷 考场作战流程

标签：考试节奏、3 小时、32 次提交、提交检查、兜底输出

一句话用途：在 3 小时机考里保持节奏，先拿确定分，再升级。

题面触发词：正式考试开始后全程使用。

什么时候用：开场扫题、每题开写、每次提交前、卡住时。

不要什么时候用：不要把时间花在重读流程本身；流程是为了减少犹豫。

复杂度：时间管理复杂度为 O(果断)。

数据范围参考：所有题。

依赖的标准容器：OPS-00、ROUTE-00、ROUTE-01。

输入如何整理：每题先按 OPS-00 标准容器整理输入。

接口：读题 -> 路由 -> 部分分 -> 升级 -> 自检 -> 提交。

输出能力：3 小时时间策略、32 次提交策略、提交前检查清单、完全不会时兜底策略。

下游可接：所有卷。

可拼接模块：所有模块。

模板代码：无。

调用示例：开场按时间表执行。

常见坑：死磕一题、第一版就追正解、提交次数被样例调试耗尽、不会时连合法输出都不写。

暴力/部分分替代：先写小数据精确解，再看是否升级。

升级方向：按分值密度升级，而不是按个人执念升级。

最小测试样例：样例、手造边界、随机小数据对拍可选。

1. 考场总原则

目标不是写出最漂亮题解。

目标是在限制时间和提交次数内，交出最多确定分。

五条铁律：

- 1. 每题先有合法输出，再追求高分。
- 2. 输入整理层只写一次，后面升级不要重写读入。
- 3. 第一版优先暴力/部分分，能过样例再升级。
- 4. 每次提交只验证一个明确变化。
- 5. 最后 5 分钟不写新算法，只做低风险修补和兜底。

2. 3 小时时间策略

按 180 分钟计算。若题数不同，比例不变。

时间段	动作	产出
0-10 分钟	全部扫题，圈 n/m/q、题型、明显部分分	给每题标 A/B/C：会、可部分、暂放
10-35 分钟	做最容易题的第一版	至少 1 题通过样例并提交
35-70 分钟	给每题补“能写出的最低合法版本”	每题至少有读入和兜底/暴力思路
70-115 分钟	攻击中档升级：数据结构、BFS/Dijkstra、背包、DP	提升已提交题分数
115-145 分钟	处理难题部分分：小数据 DFS、memo、特殊性质	不空题
145-165 分钟	回头修 WA/TLE 最明显的题	只改能解释清楚的 bug
165-175 分钟	全题提交前检查，补缺输出	防 RE/PE/漏交
175-180 分钟	冻结新功能，只做最终确认	所有题保留最稳版本

题目优先级：

会写正解且代码短 > 能拿稳定部分分 > 正解长但容易错 > 完全没路由

单题止损线：

20 分钟没有跑通样例：降级写部分分。

35 分钟没有任何提交：写合法兜底并换题。

同一题连续 3 次 WA 且原因不明：停止提交，回到本地手造样例。

3. 32 次提交策略

每题最多 32 次提交时，不要把提交当编译按钮。提交前本地至少过样例和 2 个手造点。

提交编号	用途	允许动作
1	最小可运行版本	暴力/兜底/样例通过后交
2-4	修读入、输出格式、明显边界	每次只改一个问题
5-8	部分分版本稳定	小数据 DFS、前缀和、BFS 等
9-16	正解升级	树状数组、Dijkstra、DP、线段树等
17-22	针对 WA/TLE 修复	必须写明猜测原因再改
23-26	替代路线	例如 Dijkstra 改 Floyd 小图、递推改 memo
27-30	保守修补	越界、初始化、11、方向、取模
31-32	最终保险	只提交最稳版本，不做大改

每次提交前在草稿上写一句：

这次提交只验证：_____

禁止提交：

1. 刚大改完但没跑样例。
2. 一次改了三个以上位置且不知道哪个影响结果。
3. 只是“感觉可能好了”。
4. 最后 5 分钟切换全新算法。

4. 先交部分分再升级

标准升级链：

合法输出

- > 小数据暴力
- > 记忆化/剪枝
- > 标准容器 + 主模块
- > 特判/边界修补

常见升级表：

第一版	升级版	保留不变
每次循环求区间和	PrefixSum/树状数组	read_input()、查询循环
DFS 判可达	BFS 最短路	Graph
Bellman-Ford 小数据	Dijkstra	Graph
双重循环逆序对	Compressor + 树状数组	Array
全排列访问点	BitmaskDP	距离矩阵
纯 DFS 选物品	背包 DP	物品数组
DFS DAG 路径	Topo + DP	Graph directed
每问爬树路径	LCA	Graph undirected、depth

代码结构:

```
void solve() {
    read_input();

    // 需要时保留部分分开关, 最终可手动切到 optimized.
    if (false) {
        cout << solve_bruteforce() << "\n";
    } else {
        cout << solve_optimized() << "\n";
    }
}
```

5. 提交前检查清单

60 秒快检:

类别	检查项
编译	C++17; 无 freopen; 无 #pragma; 无 #define int long long
输入	多组数据是否处理; 读入数量是否和题面一致; 字符串是否含空格
索引	图/数组是否 1-index; 字符串是否 0-index; n+1/n+2 是否够
区间	[1,r] 闭区间; r+1 是否越界保护; l>r 怎么处理
类型	距离/答案/乘法/计数是否 ll; INF 是否够大
初始化	dist/memo/vis/dp/ans 是否清空; 多测是否重新 init
图	有向/无向是否写对; 权值是否读入; 负边是否误用 Dijkstra
DP	初值是 0、INF 还是 -INF; 循环顺序是否正确
取模	加减乘是否取模; 负数是否 (x%MOD+MOD)%MOD
输出	每个询问都有输出; 大小写; 空格/换行; 无调试打印

手造最小点:

n=1
q=1
l=r
全 0 / 全相等 / 全负数
图不连通
起点等于终点
容量 W=0
没有可行解

6. 完全不会时兜底策略

完全不会也要做三件事: 读完输入、输出合法形状、避免 RE/PE。

兜底步骤:

1. 读题面输出格式, 判断输出几行、每行几个数。
2. 完整读入所有输入, 不要提前 return 导致交互/多测错位。
3. 若有 q 个询问, 循环输出 q 行。
4. 优先输出题目允许的“不存在”标记, 如 -1、NO、0。
5. 如果问最小值且允许不可达, 常见输出 -1。
6. 如果问方案数, 输出 0。

- 7. 如果问 YES/NO, 输出 NO。
- 8. 如果问构造且允许无解, 输出 NO 或 -1。
- 9. 如果必须输出一个排列, 输出 1..n。
- 10. 如果必须输出一个数组, 输出全 0 或原数组, 按题面限制选择。

常见输出类型兜底表：

输出要求	兜底输出	注意
单个答案数	0	若题面规定无解为 -1, 优先 -1
每个查询一个数	每行 0	行数必须等于查询数
YES/NO	NO	大小写按题面
是否存在并构造	NO	不要乱输出构造
最短路不可达	-1	题面若要求 INF 或别的标记按题面
方案数取模	0	合法且稳定
排列	1 2 ... n	仅在任意排列格式合法时
路径	-1 或 0	按题面 “不存在路径” 的格式

兜底不是放弃：

先交兜底防空题。

然后回到 ROUTE-00, 找最像的信号。

能写暴力就替换兜底。

能写模块就替换暴力。

7. 卡住时的 5 分钟复位

- 1. 停止改代码。
- 2. 写下当前模块输入、输出、不能处理什么。
- 3. 用 ROUTE-00 重新看：数据范围、操作类型、图边权。
- 4. 做一个 n=1 或 3 点样例手算。
- 5. 决定：修一个 bug、降级部分分、还是换题。

判断是否该换题：

读入都没写完：再给 5 分钟。

样例不过且原因不明：换题。

正解差一点但部分分已交：换题。

还有更短的题没看：换题。

ROUTE-00 第 0A 卷：题面路由表

模块编号：ROUTE-00

模块名称：第 0A 卷 题面路由表

标签：路由、关键词、数据范围、操作类型、模块选择

一句话用途：把题面信号、数据范围和操作类型快速映射到可抄模块。

题面触发词：区间、修改、查询、最短、连通、树、DAG、容量、方案数、字符串、坐标很大。

什么时候用：读完题后 3 分钟内，用本表决定第一版代码写什么。

不要什么时候用：不要在还没看数据范围时直接套高级模板。

复杂度：本表只做选择；复杂度见数据范围预算表。

数据范围参考：所有题目。

依赖的标准容器：Graph、Array、Query、State、Compressor。

输入如何整理：先圈 n/m/q/值域/是否修改/是否有边权/是否有环，再查表。

接口：路由到 build/add/range_add/query/dijkstra/topo/dp/dfs 等模块。

输出能力：给出优先模型、主模块、辅助模块和不要使用的场景。

下游可接：所有算法卷。

可拼接模块：见各表。

模板代码：无。

调用示例：按“信号 -> 模型 -> 容器 -> 模块 -> 验错”的顺序执行。

常见坑：只看关键词不看修改；只看 n 不看 q ；图有负边还用 Dijkstra；树题没确认是否真是树。

暴力/部分替代：查不到就先按数据范围写暴力或 memo。

升级方向：从预算表升级到 $O(n \log n)$ 、 $O(n)$ 或图/DP 专门模块。

最小测试样例：每条路由至少测边界、小数据、极端值。

1. 3 分钟路由动作

第 1 分钟：圈数据范围 n, m, q ，值域，写下目标复杂度。

第 2 分钟：圈题面动作：查询、修改、路径、连通、选择、计数、匹配。

第 3 分钟：查本表，决定标准容器和第一版模块。

优先级：

先看数据范围能不能承受。

再看有没有修改。

再看图边权和方向。

最后看目标：最小/最大/方案数/可行性/构造。

2. 题面信号路由表

题面信号	优先模型	标准容器	主模块	辅助模块	不要用在
数组不变，多次问 [1,r] 和	静态区间和	<code>vector<ll> a</code>	PrefixSum	Query	有修改
数组不变，多次问 min/max/gcd	静态 RMQ	<code>vector<ll> a</code>	SparseTable	Log 表	有修改、区间和
单点修改，区间和	动态前缀	Array	树状数组	SegmentTree	区间修改
单点修改，区间最值	动态区间最值	Array	SegmentTree	Compressor	只问区间和时优先 树状数组
区间加，单点查	差分	Array	Difference / 差分 树状数组	Query	还要区间和
区间加，区间和	动态区间结构	Array	LazySegTree / 双 树状数组	Compressor	只做最后输出
值域很大，但只比较 大小/排名	离散化	Compressor	树状数组 / SegmentTree	Sort	需要真实连续距离
求逆序对、前面有多少 数大/小	动态统计	Compressor	树状数组	Sort	n 很小可暴力
滑动窗口最大/最小	单调队列	Array	MonotonicQueue	Deque	窗口长度不固定且有 复杂删除
最近更大/更小、贡 献边界	单调栈	Array	MonotonicStack	前后边界数组	需要任意区间动态查 询
连通性、合并集合	并查集	边表	DSU	Kruskal	需要删除边
最小生成树、修路最 小费用	MST	<code>Graph.edges</code>	Kruskal	DSU	有向图最小树形图
无权最短路、最少步 数	BFS	Graph / Grid	BFS	queue	有权边
边权非负最短路	单源最短路	Graph	Dijkstra	priority_queue	有负边
有负边最短路	松弛最短路	<code>Graph.edges</code>	Bellman-Ford / SPFA	负环判断	n, m 很大且无负边
小图任意两点最短路	全源最短路	<code>Graph.edges</code> + 矩 阵	Floyd	路径恢复	$n > 500$ 通常危险
DAG、依赖、先后 顺序	拓扑	Graph directed	Topo	DP	有环
DAG 上最长/最 短/方案数	DAG DP	Graph directed	Topo + DP	入度	有环未处理
树上祖先、两点路径	树上查询	Graph undirected	LCA	DFS depth/dist	不是树或森林

题面信号	优先模型	标准容器	主模块	辅助模块	不要用在
树上选点、父子依赖	树形 DP	Graph undirected	DFS DP	State	一般图有环
树上选 K 个、子树合并容量	树上背包	Graph tree + State	DP-14 / DP-19	Knapsack	一般图有环
每个点作为根/中心、贡献重算	换根 DP	Graph tree	TREE-02	TreeDP	非树
选/不选、容量限制	背包	State	0/1 Knapsack	滚动数组	物品可无限选
可以无限选某物品	完全背包	State	Complete Knapsack	循环顺序	每物只能选一次
每组最多选一个	分组背包	State	Group Knapsack	分组循环	物品无组限制
两个字符串/序列匹配	双序列 DP	string / Array	LCS / EditDistance	DP 表	只需子串可考虑哈希/KMP
LIS/LCS 变体、LCIS、公共且上升	序列 DP 变体	Array / string	DP-23	树状数组/SegmentTree	连续子串要换模型
训练集、测试集、标签、模型、SPJ 得分	AI 包装题	样本表 / 特征矩阵	AI-00 / AI-10	AI-02..15	不要上第三方库或随机大模型
SVM、DNN、反向传播、自动求导	AI 公式模拟	计算图 / 矩阵向量	AI-11..15	SIM-03	数据大时先 baseline
JSON/CSV/INI、表达式、脚本规则	解析模拟	Token / AST	SIM-03/04/05	string / map	不要临场乱写半解析
日期、时区、经过天数、历法	日期模拟	Date / day number	SIM-06	数学取模	夏令时规则不明时按题面
区间合并、区间删除、括号匹配	区间 DP	Array	IntervalDP	PrefixSum	n 很大
n <= 20 且访问集合	状压	State mask	BitmaskDP / DFS	Floyd/Dijkstra	n > 22 基本爆
1..N、数位限制、上界很大	数位 DP	digits + state	DP-17	DFS memo	小范围普通枚举
dp[i]=min/max dp[j]+cost 且查询范围	DP 优化	State + query	DP-18	树状数组/SegmentTree	先写朴素
方案数、可行性、能否凑出	计数/可行性 DP	State	DP-20	MOD / bitset	不要用取模值判可达
多重/二维/价值维度/至少装满背包	背包变体	Knapsack State	DP-24	DP-06/07/08	先判至多/恰好/至少
斜率、分治、Knuth、SOS、轮廓线等高阶 DP 信号	高阶 DP 索引	State	DP-27	先交朴素/记忆化	不满足条件别硬套
重复状态明显但不好表推	记忆化搜索	State	DFS + memo	map tuple	状态有环且无处理
求第 k、小于等于 x 个数	排名/二分	Compressor	树状数组 / SegmentTree	二分答案	静态可排序二分
最大最小值、最小化最大值	二分答案	判断函数	BinarySearch + Check	Greedy/Graph/DP check	不单调

3. 数据范围预算表

先用最大量级估算，不要被样例骗。

数据范围	目标复杂度	常见可用模块	考场口令
n <= 8..10	O(n!)	全排列、爆搜	可以先全排列拿分
n <= 18..22	O(2^n n)	状压 DP、子集枚举	看到集合访问先想 mask
n <= 35..45	O(2^(n/2))	折半枚举	一分为二，排序合并
n <= 300	O(n^3)	Floyd、区间 DP	三重循环可接受
n <= 500	勉强 O(n^3)	Floyd、小规模 DP	常数要小
n <= 3000	O(n^2)	双序列 DP、普通 DP	双重循环可试
n <= 5000	O(n^2) 边缘	DP、枚举优化前版本	C++ 勉强，注意常数
n, q <= 2e5	O((n+q)log n)	树状数组、SegmentTree、Dijkstra、排序	默认 log 结构

数据范围	目标复杂度	常见可用模块	考场口令
$n, m \leq 2e5$	$O((n+m)\log n)$	BFS、Dijkstra、Kruskal、Topo	图用邻接表
$n \leq 1e6$	$O(n)$ 或 $O(n \log n)$	前缀、差分、双指针、筛法	少开二维
值域 $1e9/1e18$, 数量 $2e5$	$O(n \log n)$	Compressor + 树状数组	坐标压缩
网格 $n*m \leq 2e5$	$O(nm)$	Grid BFS/DP	二维直接做
网格 $n, m \leq 1000$	$O(nm)$	BFS、前缀、DP	内存约百万级
多测试总和给出	按总和	清空容器	不要按单测最大乘 T
多测试无总和	保守	暴力分档/剪枝	防止总量爆炸

粗算：

$1e8$ 次简单操作已经危险。

$2e8$ 次以上不要赌，除非常数极小且时间宽。

`vector<vector<ll>>(5000,5000)` 约 200MB，通常危险。

4. 操作类型路由表

操作组合	第一选择	接口	复杂度	部分分版本	升级方向
静态区间和	PrefixSum	build/query	$O(n+q)$	每问循环 $O(nq)$	无需升级
静态区间	SparseTable	build/query	$O(n\log n+q)$	每问循环	SegmentTree 若有修改
min/max/gcd	树状数组	add/prefix/query	$O(\log n)$	直接改数组再循环	SegmentTree
单点加 + 前缀/区间和	树状数组	add(pos, new-old)	$O(\log n)$	数组暴力	SegmentTree
单点赋值 + 区间和	SegmentTree	setv/query	$O(\log n)$	数组暴力	无
单点赋值 + 区间最值	SegmentTree	setv/query	$O(\log n)$	数组暴力	无
区间加 + 最后输出	Difference	diff[1]+=x, diff[r+1]-=x	$O(n+q)$	每次循环加	差分树状数组
区间加 + 单点查	差分树状数组	range_add/at	$O(\log n)$	差分离线	LazySegTree
区间加 + 区间和	LazySegTree / 双树状数组	range_add/query	$O(\log n)$	每次循环	LazySegTree
第 k 小 / 动态排名	Compressor + 树状数组	add/prefix + 二分	$O(\log^2 n)$	排序数组重算	权值线段树
连通块合并	DSU	find/unite	近似 $O(1)$	DFS 每次判连通	无删除边
加边判是否成环	DSU	unite 返回真假	近似 $O(1)$	DFS	Kruskal
无权单源最短路	BFS	bfs(G,s)	$O(n+m)$	DFS 不保证最短	Dijkstra 若有权
非负权单源最短路	Dijkstra	dijkstra(G,s)	$O((n+m)\log n)$	Bellman-Ford 小数据	无负边
任意两点小图最短路	Floyd	dist[i][j]	$O(n^3)$	每次 BFS/Dijkstra	Johnson 不作为优先
有向无环依赖	Topo	topo_sort(G)	$O(n+m)$	DFS 记忆化	DAG DP
树上路径距离	DFS + LCA	lca.query(u,v)	$O(\log n)$	每问 BFS/爬父亲	树剖低优先
容量选择最优	背包 DP	dp[j]	$O(nw)$	DFS 枚举	优化循环/滚动
复杂状态最优	Memo DFS	dfs(state)	状态数 * 转移	暴力 DFS	表推 DP

5. 路由冲突时怎么判

有修改优先看操作类型，不要直接用静态算法。

有边权优先看权值符号：无权 BFS，非负 Dijkstra，负边 Bellman-Ford/SPFA。

是树先确认 $m = n - 1$ 且连通；否则按一般图处理。

有 DAG 字样但可能有环，先 topo 检查 `order.size() == n`。

值域大但只出现 $2e5$ 个数，先 Compressor。

能先拿部分分时，先写暴力版本，保留函数名再升级。

ROUTE-01 第 0A 卷：拼接食谱

模块编号：ROUTE-01

模块名称: 第 0A 卷 拼接食谱

标签: 食谱、模块组合、部分分、升级、最小验错

一句话用途: 遇到常见题面时, 按固定顺序把标准容器和算法模块接起来。

题面触发词: 多次查询、区间修改、最短路、连通块、DAG、树上路径、容量、区间合并、访问所有点。

什么时候用: 查完 ROUTE-00 后, 用本卷抄“拼接顺序”和“最小验错”。

不要什么时候用: 不要跳过判断信号直接背食谱; 题目只要多一个修改或负边, 食谱就可能换。

复杂度: 每个食谱单独列。

数据范围参考: 按 ROUTE-00 预算表选择。

依赖的标准容器: Graph、Array、Query、State、Compressor。

输入如何整理: 先转成标准容器, 再接主模块。

接口: build/add/range_add/query/dijkstra/topo/dfs/dp。

输出能力: 给出可直接装配的代码骨架。

下游可接: C++17/STL、暴力、DP、数据结构、图论卷。

可拼接模块: 见每个食谱。

模板代码: 食谱给伪代码, 具体模板从对应算法卷抄。

调用示例: 见每个食谱。

常见坑: 判断信号不完整、忘记部分分、升级时改坏输入层、最小验错没做。

暴力/部分分替代: 每个食谱都给。

升级方向: 每个食谱都给。

最小测试样例: 每个食谱至少列 3 个风险点。

食谱格式

场景:

判断信号:

模块组合:

拼接顺序:

伪代码骨架:

部分分版本:

升级版本:

最小验错:

R01 静态区间和

场景: 多次询问数组 $[l, r]$ 的和。

判断信号: 数组读完后不再修改; q 次询问; 关键词“区间和”“子段和查询”。

模块组合: DS-01 PrefixSum。

拼接顺序:

1. 读 n, q 。
2. 读 $a[1..n]$ 。
3. PrefixSum ps; ps.build(a)。
4. 每个查询输出 ps.query(l, r)。

伪代码骨架:

```
read n, q
read a[1..n]
PrefixSum ps; ps.build(a)
repeat q:
    read l, r
    print ps.query(l, r)
```

部分分版本：每次 for $i=1..r$ 累加， $O(nr)$ 。

升级版：PrefixSum， $O(n+q)$ 。

最小验错： $l=r$ ； $l=1$ ； $r=n$ ；全负数；答案超过 int。

R02 单点修改 + 区间和

场景：两类操作：修改一个位置，查询一段和。

判断信号：op=1 pos x、op=2 l r；“单点修改”“区间查询”。

模块组合：DS-02 树状数组。

拼接顺序：

1. 读 $a[1..n]$ 。
2. 树状数组 fw；fw.build(a)。
3. 若单点加：fw.add(pos, delta)。
4. 若单点赋值：delta = newValue - a[pos]，更新 a[pos]，再 fw.add(pos, delta)。
5. 查询：fw.query(l, r)。

伪代码骨架：

```
BIT fw; fw.build(a)
if (op == 1) {
    cin >> pos >> x;
    ll delta = x - a[pos];
    a[pos] = x;
    fw.add(pos, delta);
} else {
    cin >> l >> r;
    cout << fw.query(l, r) << "\n";
}
```

部分分版本：直接改数组，查询时循环求和。

升级版：树状数组， $O((n+q)\log n)$ 。

最小验错：连续修改同一位置；查询单点；负数修改； $n=1$ 。

R03 区间加 + 最后输出

场景：很多次给 $[l, r]$ 加值，最后输出整个数组或最终状态。

判断信号：所有修改先给完，中间没有查询；关键词“最后输出”。

模块组合：DS-01 Difference。

拼接顺序：

1. 读原数组 a。
2. 建 diff: diff[i] = a[i] - a[i-1]。
3. 每次区间加 x: diff[l] += x, diff[r+1] -= x。
4. 最后前缀还原 a。

伪代码骨架：

```
vector<ll> diff(n + 2, 0);
for (int i = 1; i <= n; i++) diff[i] = a[i] - a[i - 1];
for (int qi = 0; qi < q; qi++) {
    int l, r;
    ll x;
    cin >> l >> r >> x;
    diff[l] += x;
    diff[r + 1] -= x;
}
ll cur = 0;
for (int i = 1; i <= n; i++) {
    cur += diff[i];
    cout << cur << (i == n ? '\n' : ' ');
}
```

部分分版本：每次循环 $i=1..r$ 加。

升级版：差分， $O(n+q)$ 。

最小验错： $r=n$ 时 $\text{diff}[n+1]$ 存在；负数加；区间长度 1；所有区间覆盖。

R04 区间加 + 单点查

场景：在线操作，区间加，询问某个点当前值。

判断信号：`range add l r x` 和 `query pos` 混在一起；只查单点。

模块组合：DS-02 差分树状数组。

拼接顺序：

1. 树状数组维护差分。
2. 初始数组用 `range_add(i, i, a[i])` 或先建 `diff`。
3. 区间加：`add(l, x)`, `add(r+1, -x)`。
4. 单点值：`prefix(pos)`。

伪代码骨架：

```
void range_add(int l, int r, ll x) {
    fw.add(l, x);
    if (r + 1 <= n) fw.add(r + 1, -x);
}
ll at(int pos) {
    return fw.prefix(pos);
}
```

部分分版本：直接循环加，单点输出。

升级版：差分树状数组， $O((n+q)\log n)$ 。

最小验错： $r=n$ ；多次覆盖同一点；初值不是 0；负数加。

R05 区间加 + 区间和

场景：在线区间加，还要在线查询区间和。

判断信号：操作中同时出现“区间修改”和“区间求和”。

模块组合：DS-03 LazySegmentTree 或 DS-02 双树状数组。

拼接顺序：

1. 若只求和，优先 双树状数组；若还要求 `min/max`，优先 LazySegmentTree。
2. `build(a)`。
3. 修改：`range_add(l, r, x)`。
4. 查询：`query(l, r)`。

伪代码骨架：

```
seg.build(a)
if (op == 1) seg.range_add(l, r, x);
else cout << seg.query(l, r) << "\n";
```

部分分版本：数组循环修改，查询循环求和。

升级版：LazySegmentTree， $O((n+q)\log n)$ 。

最小验错：连续区间修改后查询；查询整个数组；负数加； $l=r$ 。

R06 静态区间最值

场景：数组不变，多次询问区间最大值、最小值或 `gcd`。

判断信号：无修改； q 很大；操作可重叠合并。

模块组合：DS-03 SparseTable。

拼接顺序：

1. 读 `a`。
2. `SparseTable st; st.build(a)`。
3. 每次 `st.query(l, r)`。

伪代码骨架：

```
SparseTable st;
st.build(a);
while (q--) {
    cin >> l >> r;
    cout << st.query(l, r) << "\n";
}
```

部分分版本：每次循环找 `min/max/gcd`。

升级版本：`SparseTable`, $O(n \log n + q)$ 。

最小验错：全相等；区间长度 1；长度不是 2 的幂；不要拿 ST 做区间和。

R07 大坐标动态统计

场景：值域到 $1e9/1e18$ ，但操作次数只有 $2e5$ 左右，需要统计排名、个数、前缀数量。

判断信号：坐标/权值很大；只关心相对大小；所有操作可以先读入。

模块组合：CPP-007 Compressor + DS-02 树状数组。

拼接顺序：

1. 先读所有操作到 `queries`。
2. 把会出现的 `x` 放进 `all`。
3. `Compressor cp.build(all)`。
4. 树状数组 `fw.init(cp.size())`。
5. 按原顺序处理操作。

伪代码骨架：

```
for each query:
    all.push_back(x)
cp.build(all)
fw.init(cp.size())
for each query:
    int p = cp.id(x)
    fw.add(p, delta)
```

部分分版本：用 `vector` 存所有当前值，每次排序或循环。

升级版本：Compressor + 树状数组, $O(q \log q)$ 。

最小验错：重复值；查询范围内无值；负坐标；`lower_id > upper_id`。

R08 逆序对

场景：求数组中 $i < j$ 且 $a[i] > a[j]$ 的对数。

判断信号：关键词“逆序对”“交换排序次数”“前面比它大的数”。

模块组合：CPP-007 Compressor + DS-02 树状数组。

拼接顺序：

1. 压缩所有 `a[i]`。
2. 从左到右扫描。
3. 已出现个数 = `i - 1`。
4. 小于等于当前的个数 = `fw.prefix(id)`。
5. 前面更大的个数 = 已出现 - `fw.prefix(id)`。
6. `fw.add(id, 1)`。

伪代码骨架：

```
ll ans = 0;
for (int i = 1; i <= n; i++) {
    int p = cp.id(a[i]);
```

```

    ans += (i - 1) - fw.prefix(p);
    fw.add(p, 1);
}

```

部分分版本：双重循环 $O(n^2)$ 。

升级版本：树状数组， $O(n \log n)$ 。

最小验错：全相等答案 0；严格递增 0；严格递减 $n*(n-1)/2$ ；答案用 ll。

R09 滑动窗口最值

场景：固定长度窗口，每个窗口求最大/最小。

判断信号：关键词“连续 k 个”“窗口”“每一段长度为 k”。

模块组合：DS-04 MonotonicQueue。

拼接顺序：

1. 维护 deque 存下标。
2. 新元素进队前弹掉不可能成为答案的尾部。
3. 弹掉窗口外队首。
4. $i \geq k$ 时输出队首对应值。

伪代码骨架：

```

for (int i = 1; i <= n; i++) {
    while (!dq.empty() && a[dq.back()] <= a[i]) dq.pop_back();
    dq.push_back(i);
    while (!dq.empty() && dq.front() <= i - k) dq.pop_front();
    if (i >= k) cout << a[dq.front()] << "\n";
}

```

部分分版本：每个窗口循环找最值。

升级版本：单调队列， $O(n)$ 。

最小验错：k=1; k=n; 全相等；递增/递减数组。

R10 无权最短路

场景：边没有权，求最少边数、最少步数、最少操作次数。

判断信号：无权图；网格一步代价相同；关键词“最少步数”。

模块组合：GRAPH-02 BFS + queue。

拼接顺序：

1. 建 Graph 或保留 Grid。
2. dist 初始化 -1。
3. 起点入队 $dist[s]=0$ 。
4. 每次扩展未访问邻点。

伪代码骨架：

```

queue<int> q;
vector<int> dist(n + 1, -1);
dist[s] = 0; q.push(s);
while (!q.empty()) {
    int u = q.front(); q.pop();
    for (auto e : G.g[u]) {
        int v = e.to;
        if (dist[v] == -1) {
            dist[v] = dist[u] + 1;
            q.push(v);
        }
    }
}
}

```

部分分版本: DFS 只能判可达, 不能保证最短; 可拿连通性相关分。

升级版本: BFS, $O(n+m)$ 。

最小验错: 起点等于终点; 不可达; 重边; 网格边界。

R11 非负权最短路

场景: 边有非负权, 求从一个点到其他点的最短距离。

判断信号: 边权 $w \geq 0$; 关键词 “最短路径” “最小花费”。

模块组合: GRAPH-03 Dijkstra + CPP-004 priority_queue。

拼接顺序:

1. Graph 加权建图。
2. dist 初始化 LINF。
3. 小根堆存 {dist, node}。
4. 弹出旧状态要跳过。
5. 松弛邻边。

伪代码骨架:

```
dist[s] = 0;
pq.push({0, s});
while (!pq.empty()) {
    auto [du, u] = pq.top(); pq.pop();
    if (du != dist[u]) continue;
    for (auto e : G.g[u]) {
        if (dist[e.to] > du + e.w) {
            dist[e.to] = du + e.w;
            pq.push({dist[e.to], e.to});
        }
    }
}
```

部分分版本: Bellman-Ford 小数据 $O(nm)$ 或 DFS 枚举简单路径极小数据。

升级版本: Dijkstra, $O((n+m)\log n)$ 。

最小验错: 不可达; 多条重边; 0 权边; 有负边时不能用。

R12 小图全源最短路

场景: 任意两点最短路, $n \leq 300/500$ 。

判断信号: 多次问任意 u, v ; 图小; 可能要中转。

模块组合: GRAPH-04 Floyd。

拼接顺序:

1. dist[n+1][n+1] 初始化 LINF。
2. dist[i][i]=0。
3. 用 G.edges 初始化边。
4. 三重循环 k, i, j 。
5. 回答 dist[u][v]。

伪代码骨架:

```
for k in 1..n:
    for i in 1..n:
        for j in 1..n:
            dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j])
```

部分分版本: 每次询问跑 BFS/Dijkstra。

升级版本: Floyd, 预处理 $O(n^3)$, 查询 $O(1)$ 。

最小验错: 重边取 min; 不可达; 有向/无向初始化; LINF + LINF 防溢出。

R13 最小生成树

场景：连接所有点的最小总费用。

判断信号：无向图；关键词“修路”“连接所有城市”“总费用最小”。

模块组合：GRAPH-06 DSU + Kruskal。

拼接顺序：

1. 用 Graph 读边。
2. 复制 G.edges 并按 w 升序排序。
3. DSU.init(n)。
4. 能 unite 的边加入答案。
5. 最后检查加入边数是否 n-1。

伪代码骨架：

```
sort(es by w)
for e in es:
    if dsu.unite(e.from, e.to):
        ans += e.w
        cnt++
if cnt != n - 1: no solution
```

部分分版本：枚举边集只适合极小数据；或只判连通拿部分分。

升级版本：Kruskal, $O(m \log m)$ 。

最小验错：图不连通；重边；负权边也可用；不要用有向图。

R14 DAG 依赖顺序 + DP

场景：任务依赖、课程先修、从前驱转移到后继。

判断信号：有向边表示先后；题面说“无环”或可拓扑；求最长链/方案数/最早完成时间。

模块组合：GRAPH-05 Topo + DAG DP。

拼接顺序：

1. 用 `G.add\_directed(u, v)` 建图。
2. topo_sort 得 order。
3. 若 order.size()!=n, 说明有环。
4. 按 order 顺序转移 dp。

伪代码骨架：

```
vector<int> order = topo_sort(G);
for (int u : order) {
    for (auto e : G.g[u]) {
        int v = e.to;
        dp[v] = max(dp[v], dp[u] + value);
    }
}
```

部分分版本：DFS + memo, 遇到正在访问的点标记有环。

升级版本：Topo + DP, $O(n+m)$ 。

最小验错：多个入度 0；有环；不连通 DAG；方案数取模。

R15 树上路径查询

场景：树上两点距离、最近公共祖先、路径信息。

判断信号：n 个点 n-1 条边；无向连通；多次问 u, v。

模块组合：GRAPH-09 LCA。

拼接顺序：

1. 无向建树。
2. 从根 1 DFS, 得到 depth、up、distRoot。

3. 每问 $lca = query(u, v)$ 。
4. 距离 $= distRoot[u] + distRoot[v] - 2 * distRoot[lca]$ 。

伪代码骨架：

```
lca.build(G, 1);
int z = lca.query(u, v);
ll d = distRoot[u] + distRoot[v] - 2 * distRoot[z];
```

部分分版本：每次 BFS/DFS 找父亲路径， $O(nq)$ 。

升级版：LCA，预处理 $O(n \log n)$ ，查询 $O(\log n)$ 。

最小验错： $u=v$ ；根参与查询；链状树；星状树；边权为 0。

R16 0/1 背包

场景：每个物品最多选一次，在容量内最大价值/可行性/方案数。

判断信号：“每个只能选一次” “容量 W ” “重量/价值”。

模块组合：DP-06/07/08/24 Knapsack。

拼接顺序：

1. $dp[0..W]$ 初始化。
2. 枚举物品 i 。
3. 容量 j 从 W 倒序到 $weight[i]$ 。
4. $dp[j] = \max(dp[j], dp[j-w]+val)$ 。

伪代码骨架：

```
for (int i = 1; i <= n; i++) {
    for (int j = W; j >= w[i]; j--) {
        dp[j] = max(dp[j], dp[j - w[i]] + val[i]);
    }
}
```

部分分版本：DFS 枚举选/不选。

升级版：滚动数组 DP， $O(nW)$ 。

最小验错： $w=0$ ；物品重量大于 W ；价值为 0；倒序循环不能写反。

R17 区间 DP

场景：区间合并、括号匹配、删除一段、石子合并。

判断信号：答案定义在 $[l, r]$ ；转移枚举中间点 k ； $n \leq 300/500$ 。

模块组合：DP-13 Interval DP + DS-01 PrefixSum。

拼接顺序：

1. 初始化长度为 1 的区间。
2. 枚举 len 从 2 到 n 。
3. 枚举 l ，得到 r 。
4. 枚举分割点 k 。
5. 若需要区间和，用 $PrefixSum.query(l, r)$ 。

伪代码骨架：

```
for (int len = 2; len <= n; len++) {
    for (int l = 1; l + len - 1 <= n; l++) {
        int r = l + len - 1;
        for (int k = l; k < r; k++) {
            dp[l][r] = min(dp[l][r], dp[l][k] + dp[k + 1][r] + cost(l, r));
        }
    }
}
```

部分分版本：DFS 枚举合并顺序 + memo。

升级版：表推区间 DP， $O(n^3)$ 。

最小验错：长度 1；长度 2；初始化 INF；区间和下标。

R18 访问所有点状压

场景：从若干点中访问所有点，求最短路径/最小代价。

判断信号：n <= 20；关键词“每个点访问一次/至少一次”“集合状态”。

模块组合：GRAPH-03/04 shortest path + DP-16 Bitmask DP。

拼接顺序：

1. 先求关键点两两距离。
2. dp[mask][last] 表示访问集合 mask，最后在 last 的最小代价。
3. 枚举 mask、last、next。
4. 答案取完整 mask 的最小值。

伪代码骨架：

```
dp[1 << (start - 1)][start] = 0; // start 是 1..k 的关键点编号, full = 1 << k
for (int mask = 0; mask < full; mask++) {
    for (int last = 1; last <= k; last++) {
        if (dp[mask][last] == INF) continue;
        for (int nxt = 1; nxt <= k; nxt++) {
            if (mask >> (nxt - 1) & 1) continue;
            int nmask = mask | (1 << (nxt - 1));
            dp[nmask][nxt] = min(dp[nmask][nxt], dp[mask][last] + dist[last][nxt]);
        }
    }
}
```

部分分版本：全排列枚举访问顺序。

升级版本：状压 DP， $O(2^n \cdot n^2)$ 。

最小验错：不可达距离；起点是否固定；回不回起点； $1 < n$ 溢出，n 不要太大。

R19 复杂状态 DFS + memo

场景：题目选择多、状态难表推，但递归会重复。

判断信号：暴力 DFS 会多次遇到同一个（进度，资源，记忆）；数据不够支持纯暴力。

模块组合：BRUTE-09 map<tuple<...>, ans> 或 DP-25 DFS + memo。

拼接顺序：

1. 写纯 dfs，参数只放影响未来的量。
2. 找 base case。
3. 在函数开头查 memo。
4. 枚举选择并更新 ans。
5. 返回前写 memo。

伪代码骨架：

```
ll dfs(int i, int rem, int last) {
    if (rem < 0) return -LINF;
    auto key = make_tuple(i, rem, last);
    if (memo.count(key)) return memo[key];
    if (i == n + 1) return memo[key] = 0;
    ll ans = dfs(i + 1, rem, last);
    if (can_choose) ans = max(ans, gain + dfs(i + 1, rem - cost, i)); // can_choose 必须包含资源可行性
    return memo[key] = ans;
}
```

部分分版本：纯 DFS。

升级版本：Memo；若范围清楚，再改数组 DP。

最小验错：base 是否写入 memo；状态是否漏了影响未来的变量；有环状态要加 visiting；返回值单位元。

R20 二分答案 + 检查函数

场景：最大化最小值、最小化最大值、求最少时间/最小容量。

判断信号：问“最小的最大”“最大的最小”；答案有单调性。

模块组合：DS-06 BinarySearch + Check + GREEDY/GRAPH/DP。

拼接顺序：

1. 明确 `check(x)` 表示 x 是否可行。
2. 证明口头单调： x 变大更容易，或 x 变大更难。
3. 设定答案左右边界。
4. 二分并调用 `check`。
5. 输出边界。

伪代码骨架：

```
while (l < r) {
    ll mid = l + (r - l) / 2;
    if (check(mid)) r = mid;
    else l = mid + 1;
}
cout << l << "\n";
```

部分分版本：枚举答案逐个 `check`。

升级版本：二分答案，复杂度 $O(\text{check} * \log V)$ 。

最小验错：边界是否包含答案；`mid` 是否溢出；`check` 单调方向；输出 `l` 还是 `r`。

v0.2 本卷例题训练区

这一节是 0.2 新增的实战例题。每题都配完整可运行代码和样例；考试时优先看“覆盖模块”和“考场用途”，再复制对应代码骨架。

V00-EX01 静态区间和路由

- 归属卷：第 0 卷
- 覆盖模块：ROUTE-00、DS-01、OPS-00
- 考场用途：训练看到“数组不修改，多次问区间和”时立即路由到前缀和。

题目描述：给定长度为 n 的整数数组，回答 q 次闭区间 $[l, r]$ 的元素和。数组不会修改。

输入格式：第一行两个整数 n q 。第二行 n 个整数。接下来 q 行，每行两个整数 l r 。

输出格式：每次询问输出一行区间和。

样例输入：

```
5 3
1 -2 3 4 5
1 3
2 5
4 4
```

样例输出：

```
2
10
4
```

完整代码：

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, q;
```

```

cin >> n >> q;
vector<ll> prefix(n + 1, 0);
for (int i = 1; i <= n; i++) {
    ll x;
    cin >> x;
    prefix[i] = prefix[i - 1] + x;
}
while (q--) {
    int l, r;
    cin >> l >> r;
    cout << prefix[r] - prefix[l - 1] << '\n';
}
return 0;
}

```

测试设计:

- 测试 1 输入:

```

1 2
7
1 1
1 1

```

期望输出:

```

7
7

```

- 测试 2 输入:

```

3 1
-5 -6 -7
1 3

```

期望输出:

```

-18

```

V00-EX02 离线区间加最终数组

- 归属卷: 第 0 卷
- 覆盖模块: ROUTE-00、DS-01、OPS-00
- 考场用途: 训练 “区间加但只在最后输出” 路由到差分数组。

题目描述: 给定数组, 执行 q 次操作: 把闭区间 $[l, r]$ 内所有数加上 x 。所有操作结束后输出最终数组。

输入格式: 第一行两个整数 n q 。第二行 n 个整数。接下来 q 行, 每行 l r x 。

输出格式: 输出一行 n 个整数, 表示最终数组。

样例输入:

```

5 3
1 2 3 4 5
1 3 2
2 5 -1
5 5 10

```

样例输出:

```

3 3 4 3 14

```

完整代码:

```

#include <bits/stdc++.h>
using namespace std;
using ll = long long;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
}

```

```

int n, q;
cin >> n >> q;
vector<ll> a(n + 1), diff(n + 2, 0);
for (int i = 1; i <= n; i++) cin >> a[i];
while (q--) {
    int l, r;
    ll x;
    cin >> l >> r >> x;
    diff[l] += x;
    diff[r + 1] -= x;
}
ll add = 0;
for (int i = 1; i <= n; i++) {
    add += diff[i];
    if (i > 1) cout << ' ';
    cout << a[i] + add;
}
cout << '\n';
return 0;
}

```

测试设计:

- 测试 1 输入:

```

4 1
0 0 0 0
1 4 5

```

期望输出:

```

5 5 5 5

```

- 测试 2 输入:

```

3 2
1 1 1
1 2 -3
2 3 4

```

期望输出:

```

-2 2 5

```

V00-EX03 单点赋值与区间和

- 归属卷: 第 0 卷
- 覆盖模块: ROUTE-00、DS-02、OPS-00
- 考场用途: 训练“单点赋值加区间和”路由到 树状数组, 并把赋值转换成差量。

题目描述: 维护一个数组, 支持两种操作: S p x 表示把 a[p] 赋值为 x; Q l r 表示查询 [l,r] 的和。

输入格式: 第一行两个整数 n q。第二行 n 个整数。接下来 q 行, 每行一个操作。

输出格式: 每个 Q 操作输出一行答案。

样例输入:

```

5 5
1 2 3 4 5
Q 1 5
S 3 10
Q 2 4
S 1 -1
Q 1 3

```

样例输出:

15
16
11

完整代码:

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

struct BIT {
    int n;
    vector<ll> bit;

    void init(int n_) {
        n = n_;
        bit.assign(n + 1, 0);
    }

    void add(int pos, ll delta) {
        for (int i = pos; i <= n; i += i & -i) bit[i] += delta;
    }

    ll prefix(int pos) const {
        ll res = 0;
        for (int i = pos; i > 0; i -= i & -i) res += bit[i];
        return res;
    }

    ll query(int l, int r) const {
        return prefix(r) - prefix(l - 1);
    }
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, q;
    cin >> n >> q;
    vector<ll> a(n + 1);
    BIT fw;
    fw.init(n);
    for (int i = 1; i <= n; i++) {
        cin >> a[i];
        fw.add(i, a[i]);
    }
    while (q--) {
        char op;
        cin >> op;
        if (op == 'S') {
            int p;
            ll x;
            cin >> p >> x;
            fw.add(p, x - a[p]);
            a[p] = x;
        } else {
            int l, r;
            cin >> l >> r;
            cout << fw.query(l, r) << '\n';
        }
    }
    return 0;
}
```

测试设计:

- 测试 1 输入:

```
1 4
5
Q 1 1
S 1 7
S 1 -2
Q 1 1
```

期望输出:

```
5
-2
```

- 测试 2 输入:

```
3 2
1 2 3
Q 1 1
Q 3 3
```

期望输出:

```
1
3
```

V00-EX04 大坐标频次统计

- 归属卷: 第 0 卷
- 覆盖模块: ROUTE-00、CPP-007、DS-02
- 考场用途: 训练 “值域巨大但出现值有限” 路由到离散化加 树状数组。

题目描述: 依次处理 q 个操作: $A\ x$ 表示加入一个值为 x 的数, 可重复加入; $C\ l\ r$ 表示询问当前数中有多少个值落在 $[l, r]$ 。

输入格式: 第一行一个整数 q 。接下来 q 行, 每行一个操作。

输出格式: 每个 C 操作输出一行答案。

样例输入:

```
6
A 1000000000
A -5
C -10 100
A 7
C 7 1000000000
C 8 9
```

样例输出:

```
1
2
0
```

完整代码:

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

struct BIT {
    int n;
    vector<int> bit;

    void init(int n_) {
        n = n_;
        bit.assign(n + 1, 0);
    }

    void add(int pos, int delta) {
```

```

        for (int i = pos; i <= n; i += i & -i) bit[i] += delta;
    }

    int prefix(int pos) const {
        int res = 0;
        for (int i = pos; i > 0; i -= i & -i) res += bit[i];
        return res;
    }
};

struct Operation {
    char type;
    ll x;
    ll y;
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int q;
    cin >> q;
    vector<Operation> ops(q + 1);
    vector<ll> coords;
    for (int i = 1; i <= q; i++) {
        cin >> ops[i].type >> ops[i].x;
        if (ops[i].type == 'A') {
            ops[i].y = 0;
            coords.push_back(ops[i].x);
        } else {
            cin >> ops[i].y;
        }
    }
    sort(coords.begin(), coords.end());
    coords.erase(unique(coords.begin(), coords.end()), coords.end());

    BIT fw;
    fw.init((int)coords.size());
    for (int i = 1; i <= q; i++) {
        if (ops[i].type == 'A') {
            int id = int(lower_bound(coords.begin(), coords.end(), ops[i].x) - coords.begin()) + 1;
            fw.add(id, 1);
        } else {
            int right = int(upper_bound(coords.begin(), coords.end(), ops[i].y) - coords.begin());
            int left = int(lower_bound(coords.begin(), coords.end(), ops[i].x) - coords.begin());
            cout << fw.prefix(right) - fw.prefix(left) << '\n';
        }
    }
    return 0;
}

```

测试设计:

- 测试 1 输入:

```

4
A 5
A 5
C 5 5
C 6 7

```

期望输出:

```

2
0

```

- 测试 2 输入:

```
3
A -100
A 0
C -200 -1
```

期望输出:

```
1
```

V00-EX05 网格最少步数

- 归属卷: 第 0 卷
- 覆盖模块: ROUTE-00、GRAPH-02、OPS-00
- 考场用途: 训练“无权最少步数”路由到 BFS, 而不是 DFS。

题目描述: 给定 n 行 m 列网格, $.$ 可走, $\#$ 不可走, S 是起点, T 是终点。每步可向上下左右走一格, 求最少步数。不可达输出 -1 。

输入格式: 第一行两个整数 n m 。接下来 n 行, 每行一个长度为 m 的字符串。

输出格式: 输出最少步数。

样例输入:

```
3 4
S..#
.#..
..T.
```

样例输出:

```
4
```

完整代码:

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    cin >> n >> m;
    vector<string> grid(n + 1);
    pair<int, int> start = {-1, -1};
    pair<int, int> target = {-1, -1};
    for (int i = 1; i <= n; i++) {
        string row;
        cin >> row;
        grid[i] = " " + row;
        for (int j = 1; j <= m; j++) {
            if (grid[i][j] == 'S') start = {i, j};
            if (grid[i][j] == 'T') target = {i, j};
        }
    }

    vector<vector<int>> dist(n + 1, vector<int>(m + 1, -1));
    queue<pair<int, int>> q;
    dist[start.first][start.second] = 0;
    q.push(start);
    int dx[4] = {1, -1, 0, 0};
    int dy[4] = {0, 0, 1, -1};
    while (!q.empty()) {
        auto [x, y] = q.front();
        q.pop();
        for (int d = 0; d < 4; d++) {
            int nx = x + dx[d];
```



```

        int ny = y + dy[d];
        if (nx < 1 || nx > n || ny < 1 || ny > m) continue;
        if (grid[nx][ny] == '#') continue;
        if (dist[nx][ny] != -1) continue;
        dist[nx][ny] = dist[x][y] + 1;
        q.push({nx, ny});
    }
}
cout << dist[target.first][target.second] << '\n';
return 0;
}

```

测试设计:

- 测试 1 输入:

```

1 2
ST

```

期望输出:

```

1

```

- 测试 2 输入:

```

1 3
S#T

```

期望输出:

```

-1

```

V00-EX06 非负权道路最短路

- 归属卷: 第 0 卷
- 覆盖模块: ROUTE-00、GRAPH-03、CPP-004
- 考场用途: 训练“非负边权最短路”路由到 Dijkstra 和小根堆。

题目描述: 给定无向非负权图和起点 s , 回答若干目标点的最短距离。不可达输出 -1 。

输入格式: 第一行三个整数 n m s 。接下来 m 行为 u v w 。然后一行整数 q 。接下来 q 行每行一个目标点。

输出格式: 每个目标点输出一行距离。

样例输入:

```

4 5 1
1 2 5
1 3 2
3 2 1
2 4 2
3 4 10
3
2
4
1

```

样例输出:

```

3
5
0

```

完整代码:

```

#include <bits/stdc++.h>
using namespace std;
using ll = long long;
const ll INF = 4'000'000'000'000'000'000LL;

int main() {
    ios::sync_with_stdio(false);

```

```

cin.tie(nullptr);

int n, m, s;
cin >> n >> m >> s;
vector<vector<pair<int, ll>>> graph(n + 1);
for (int i = 1; i <= m; i++) {
    int u, v;
    ll w;
    cin >> u >> v >> w;
    graph[u].push_back({v, w});
    graph[v].push_back({u, w});
}

vector<ll> dist(n + 1, INF);
priority_queue<pair<ll, int>, vector<pair<ll, int>>, greater<pair<ll, int>>> pq;
dist[s] = 0;
pq.push({0, s});
while (!pq.empty()) {
    auto [du, u] = pq.top();
    pq.pop();
    if (du != dist[u]) continue;
    for (auto [v, w] : graph[u]) {
        if (dist[v] > du + w) {
            dist[v] = du + w;
            pq.push({dist[v], v});
        }
    }
}

int q;
cin >> q;
while (q--) {
    int t;
    cin >> t;
    cout << (dist[t] == INF ? -1 : dist[t]) << '\n';
}
return 0;
}

```

测试设计:

- 测试 1 输入:

```

3 1 1
1 2 5
1
3

```

期望输出:

```
-1
```

- 测试 2 输入:

```

2 2 1
1 2 10
1 2 3
1
2

```

期望输出:

```
3
```

V00-EX07 依赖任务排序

- 归属卷: 第 0 卷
- 覆盖模块: ROUTE-00、GRAPH-05、OPS-00

- 考场用途：训练“先后依赖”路由到拓扑排序，并检查有环。

题目描述：有 n 个任务和 m 条依赖关系 $u\ v$ ，表示任务 u 必须在任务 v 前完成。输出字典序尽量小的合法顺序；如果不存在，输出 CYCLE。

输入格式：第一行两个整数 $n\ m$ 。接下来 m 行，每行两个整数 $u\ v$ 。

输出格式：合法时输出一行任务顺序；否则输出 CYCLE。

样例输入：

```
4 3
1 3
2 3
3 4
```

样例输出：

```
1 2 3 4
```

完整代码：

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    cin >> n >> m;
    vector<vector<int>> graph(n + 1);
    vector<int> indeg(n + 1, 0);
    for (int i = 1; i <= m; i++) {
        int u, v;
        cin >> u >> v;
        graph[u].push_back(v);
        indeg[v]++;
    }

    priority_queue<int, vector<int>, greater<int>> pq;
    for (int i = 1; i <= n; i++) {
        if (indeg[i] == 0) pq.push(i);
    }

    vector<int> order;
    while (!pq.empty()) {
        int u = pq.top();
        pq.pop();
        order.push_back(u);
        for (int v : graph[u]) {
            indeg[v]--;
            if (indeg[v] == 0) pq.push(v);
        }
    }

    if ((int)order.size() != n) {
        cout << "CYCLE\n";
    } else {
        for (int i = 0; i < n; i++) {
            if (i) cout << ' ';
            cout << order[i];
        }
        cout << '\n';
    }
    return 0;
}
```

测试设计：

- 测试 1 输入:

```
3 3
1 2
2 3
3 1
```

期望输出:

```
CYCLE
```

- 测试 2 输入:

```
3 1
2 3
```

期望输出:

```
1 2 3
```

V00-EX08 容量选择路线

- 归属卷: 第 0 卷
- 覆盖模块: ROUTE-00、DP-06、OPS-00
- 考场用途: 训练 “每个物品最多选一次” 路由到 0/1 背包, 容量循环倒序。

题目描述: 有 n 个物品, 每个物品有重量 w 和价值 v , 每个物品最多选一次。背包容量为 W , 求最大总价值。

输入格式: 第一行两个整数 n W 。接下来 n 行, 每行两个整数 w v 。

输出格式: 输出最大价值。

样例输入:

```
4 7
3 4
4 5
2 3
3 7
```

样例输出:

```
12
```

完整代码:

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, W;
    cin >> n >> W;
    vector<ll> dp(W + 1, 0);
    for (int i = 1; i <= n; i++) {
        int w;
        ll v;
        cin >> w >> v;
        for (int cap = W; cap >= w; cap--) {
            dp[cap] = max(dp[cap], dp[cap - w] + v);
        }
    }
    cout << dp[W] << '\n';
    return 0;
}
```

测试设计:

- 测试 1 输入:

1 2

1 1

期望输出:

1

- 测试 2 输入:

2 3

4 10

5 20

期望输出:

0

V00-EX09 模式串出现次数

- 归属卷: 第 0 卷
- 覆盖模块: ROUTE-00、STR-02、CPP-011
- 考场用途: 训练“长文本查模式串”路由到 KMP, 并处理重叠出现。

题目描述: 给定文本串 s 和模式串 p , 统计 p 在 s 中出现了多少次, 允许重叠。

输入格式: 第一行一个字符串 s 。第二行一个字符串 p 。

输出格式: 输出出现次数。

样例输入:

aaaaa

aa

样例输出:

4

完整代码:

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    string s_raw, p_raw;
    cin >> s_raw >> p_raw;
    string s = " " + s_raw;
    string p = " " + p_raw;
    int n = (int)s_raw.size();
    int m = (int)p_raw.size();
    vector<int> pi(m + 1, 0);
    for (int i = 2; i <= m; i++) {
        int j = pi[i - 1];
        while (j > 0 && p[i] != p[j + 1]) j = pi[j];
        if (p[i] == p[j + 1]) j++;
        pi[i] = j;
    }

    int ans = 0;
    int j = 0;
    for (int i = 1; i <= n; i++) {
        while (j > 0 && s[i] != p[j + 1]) j = pi[j];
        if (s[i] == p[j + 1]) j++;
        if (j == m) {
            ans++;
            j = pi[j];
        }
    }
}
```

```

    cout << ans << '\n';
    return 0;
}

```

测试设计:

- 测试 1 输入:

```

abc
d

```

期望输出:

```

0

```

- 测试 2 输入:

```

abababa
aba

```

期望输出:

```

3

```

V00-EX10 提交版本选择器

- 归属卷: 第 0 卷
- 覆盖模块: OPS-01、ROUTE-01、TRAIN-00
- 考场用途: 训练 “先保分, 再修正解” 的提交路线选择。

题目描述: 你有 k 个候选提交版本。每个版本有预计得分 $score$ 、风险值 $risk$ 和剩余调试时间 $time$ 。只考虑 $risk \leq R$ 且 $time \leq M$ 的版本。在可选版本中, 优先选得分最高; 得分相同选风险更低; 仍相同选用时更短; 仍相同选编号更小。若没有可选版本, 输出 HOLD。

输入格式: 第一行三个整数 k R M 。接下来 k 行, 每行三个整数 $score$ $risk$ $time$ 。

输出格式: 可选时输出版本编号和预计得分; 不可选时输出 HOLD。

样例输入:

```

4 30 20
60 10 8
80 40 12
80 25 25
70 20 18

```

样例输出:

```

4 70

```

完整代码:

```

#include <bits/stdc++.h>
using namespace std;

struct Version {
    int id;
    int score;
    int risk;
    int time_need;
};

bool better(const Version &a, const Version &b) {
    if (a.score != b.score) return a.score > b.score;
    if (a.risk != b.risk) return a.risk < b.risk;
    if (a.time_need != b.time_need) return a.time_need < b.time_need;
    return a.id < b.id;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

```

```

int k, R, M;
cin >> k >> R >> M;
bool has = false;
Version best{0, 0, 0, 0};
for (int i = 1; i <= k; i++) {
    Version cur;
    cur.id = i;
    cin >> cur.score >> cur.risk >> cur.time_need;
    if (cur.risk > R || cur.time_need > M) continue;
    if (!has || better(cur, best)) {
        best = cur;
        has = true;
    }
}

if (!has) {
    cout << "HOLD\n";
} else {
    cout << best.id << ' ' << best.score << '\n';
}
return 0;
}

```

测试设计:

- 测试 1 输入:

```

2 10 5
100 20 1
80 5 6

```

期望输出:

HOLD

- 测试 2 输入:

```

2 50 50
90 30 10
90 20 20

```

期望输出:

2 90

第 7 卷: 调试、反例与对拍训练

V00-CEX01 数据范围路由卡

- 归属卷: 第 0 卷
- 覆盖模块: ROUTE-00、复杂度表、题型信号
- 考场用途: 把题面关键词和数据范围直接映射到第一本该翻的书。
- 参考题型来源: 参考来源: 洛谷官方题单的基础/进阶分类、OI Wiki 算法分类。

题目描述: 给出若干组 $n \ m$ feature, 输出建议优先翻的模块。

输入格式: 多行, 每行 $n \ m$ feature, 读到 EOF。

输出格式: 每行输出一个模块建议。

样例输入:

```

5 4 unweighted_graph
200000 300000 weighted_nonnegative
18 0 subset
100 1000 capacity

```

样例输出:

```

GRAPH-02 BFS
GRAPH-03 Dijkstra

```

BRUTE-02 Bitmask

DP-06 Knapsack

完整代码:

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    long long n, m;
    string feature;
    while (cin >> n >> m >> feature) {
        if (feature == "unweighted_graph") cout << "GRAPH-02 BFS\n";
        else if (feature == "weighted_nonnegative") cout << "GRAPH-03 Dijkstra\n";
        else if (feature == "range_sum_static") cout << "DS-01 PrefixSum\n";
        else if (feature == "range_update") cout << "DS-01 Difference\n";
        else if (n <= 20 && feature == "subset") cout << "BRUTE-02 Bitmask\n";
        else if (feature == "capacity") cout << "DP-06 Knapsack\n";
        else cout << "ROUTE-00 Read constraints again\n";
    }
    return 0;
}
```

测试设计: 额外测试: 构造最小规模、重复值、边界值各一组, 和样例一起运行。

V00-CEX02 先交差分部分分

- 归属卷: 第 0 卷
- 覆盖模块: DS-01 差分、提交策略
- 考场用途: 看到大量区间加, 先写差分拿稳分。
- 参考题型来源: 参考来源: 洛谷入门数组/前缀差分题型。

题目描述: 长度为 n 的数组初始全 0, 执行区间加, 输出最终最大值第一次出现的位置和值。

输入格式: 第一行 n q , 之后 q 行 l r v 。

输出格式: 输出 pos $maxValue$ 。

样例输入:

```
5 3
1 3 2
2 5 1
4 4 10
```

样例输出:

```
4 11
```

完整代码:

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    vector<long long> diff(n + 3, 0);
    for (int i = 1; i <= q; i++) {
        int l, r;
        long long v;
        cin >> l >> r >> v;
    }
}
```



```

        diff[l] += v;
        diff[r + 1] -= v;
    }
    long long cur = 0, mx = LLONG_MIN;
    int pos = 1;
    for (int i = 1; i <= n; i++) {
        cur += diff[i];
        if (cur > mx) {
            mx = cur;
            pos = i;
        }
    }
    cout << pos << ' ' << mx << '\n';
    return 0;
}

```

测试设计：额外测试：构造最小规模、重复值、边界值各一组，和样例一起运行。

V00-CEX03 BFS 与 Dijkstra 路由合并

- 归属卷：第 0 卷
- 覆盖模块：GRAPH-02 BFS、GRAPH-03 Dijkstra
- 考场用途：同一份题面先判断边权是否全 1，再决定翻哪本图论页。
- 参考题型来源：参考来源：洛谷图论题单最短路分类、OI Wiki 最短路。

题目描述：给无向图，如果所有边权都是 1 用 BFS，否则用 Dijkstra，求 s 到 t 最短距离。

输入格式：第一行 n m s t，之后 m 行 u v w。

输出格式：输出最短距离，不可达输出 -1。

样例输入：

```

4 4 1 4
1 2 1
2 4 1
1 3 5
3 4 1

```

样例输出：

```

2

```

完整代码：

```

#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, s, t;
    cin >> n >> m >> s >> t;
    vector<vector<pair<int,int>>> g(n + 1);
    bool all_one = true;
    for (int i = 1; i <= m; i++) {
        int u, v, w;
        cin >> u >> v >> w;
        g[u].push_back({v, w});
        g[v].push_back({u, w});
        if (w != 1) all_one = false;
    }
    const long long INF = (long long)4e18;
    vector<long long> dist(n + 1, INF);
    if (all_one) {
        queue<int> q;
    }
}

```

```

    dist[s] = 0;
    q.push(s);
    while (!q.empty()) {
        int u = q.front(); q.pop();
        for (auto [v, w] : g[u]) {
            if (dist[v] == INF) {
                dist[v] = dist[u] + 1;
                q.push(v);
            }
        }
    }
} else {
    priority_queue<pair<long long,int>, vector<pair<long long,int>>, greater<pair<long long,int>>> pq;
    dist[s] = 0;
    pq.push({0, s});
    while (!pq.empty()) {
        auto [d, u] = pq.top(); pq.pop();
        if (d != dist[u]) continue;
        for (auto [v, w] : g[u]) {
            if (dist[v] > d + w) {
                dist[v] = d + w;
                pq.push({dist[v], v});
            }
        }
    }
}
cout << (dist[t] == INF ? -1 : dist[t]) << '\n';
return 0;
}

```

测试设计：额外测试：构造最小规模、重复值、边界值各一组，和样例一起运行。

V00-CEX04 小数据暴力与大数据特判

- 归属卷：第 0 卷
- 覆盖模块：BRUTE 子集、哈希表、部分分策略
- 考场用途：同一题先写小数据精确，再给大数据特殊版。
- 参考题型来源：参考来源：洛谷搜索/哈希题型、部分分赛制策略。

题目描述：若 $n \leq 25$ ，统计子集和等于目标的方案数；否则只统计两数和等于目标的对数，模拟部分分兜底。

输入格式：第一行 n target，第二行 n 个整数。

输出格式：输出统计结果。

样例输入：

```

4 5
1 2 3 4

```

样例输出：

```

2

```

完整代码：

```

#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    long long target;
    cin >> n >> target;
    vector<long long> a(n + 1);

```

```

for (int i = 1; i <= n; i++) cin >> a[i];
long long ans = 0;
if (n <= 25) {
    for (int mask = 0; mask < (1 << n); mask++) {
        long long sum = 0;
        for (int i = 1; i <= n; i++) if (mask & (1 << (i - 1))) sum += a[i];
        if (sum == target) ans++;
    }
} else {
    unordered_map<long long, long long> cnt;
    cnt.reserve(n * 2 + 10);
    for (int i = 1; i <= n; i++) {
        ans += cnt[target - a[i]];
        cnt[a[i]]++;
    }
}
cout << ans << '\n';
return 0;
}

```

测试设计：额外测试：构造最小规模、重复值、边界值各一组，和样例一起运行。

V00-CEX05 最高分提交统计

- 归属卷：第 0 卷
- 覆盖模块：考试策略、最高分提交规则、数组
- 考场用途：把每题多次提交取最高的规则变成程序，强化先交部分分。
- 参考题型来源：参考来源：本次机考规则。

题目描述：有 p 道题、 s 次提交记录，每条记录是题号和得分，输出最终总分。

输入格式：第一行 p s ，之后 s 行 id $score$ 。

输出格式：输出最终总分。

样例输入：

```

3 6
1 20
2 30
1 50
3 10
2 25
3 80

```

样例输出：

```

160

```

完整代码：

```

#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int p, s;
    cin >> p >> s;
    vector<int> best(p + 1, -1);
    for (int i = 1; i <= s; i++) {
        int id, score;
        cin >> id >> score;
        best[id] = max(best[id], score);
    }
    int total = 0;

```

```
for (int i = 1; i <= p; i++) total += max(0, best[i]);  
cout << total << '\n';  
return 0;  
}
```

测试设计：额外测试：构造最小规模、重复值、边界值各一组，和样例一起运行。
